

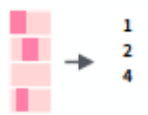
Xử lý ký tự với stringr : : CHEAT SHEET

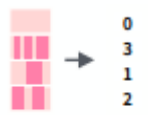


Thư viện stringr cung cấp các công cụ hữu ích để làm việc với kiểu dữ liệu định dạng ký tự.

Tìm kiếm ký tự phù hợp

 **str_detect**(string, **pattern**) Kiểm tra chuỗi có chứa ký tự cho trước.
`str_detect(fruit, "a")`

 **str_which**(string, **pattern**) Tìm vị trí của chuỗi trong vector chứa ký tự cho trước.
`str_which(fruit, "a")`

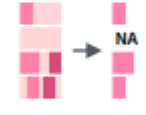
 **str_count**(string, **pattern**) Đếm ký tự cho trước trong chuỗi.
`str_count(fruit, "a")`

 **str_locate**(string, **pattern**) Tìm vị trí chính xác của chuỗi có chứa ký tự cho trước. Xem **str_locate_all**.
`str_locate(fruit, "a")`

Trích xuất ký tự

 **str_sub**(string, start = 1L, end = -1L) Trích xuất ký tự theo vị trí.
`str_sub(fruit, 1, 3); str_sub(fruit, -2)`

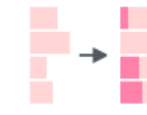
 **str_subset**(string, **pattern**) Trả ra chuỗi trong vector có chứa ký tự cho trước.
`str_subset(fruit, "b")`

 **str_extract**(string, **pattern**) Trả ra ký tự đầu tiên phù hợp với điều kiện lọc dưới dạng vector. Xem thêm **str_extract_all**.
`str_extract(fruit, "[aeiou]")`

 **str_match**(string, **pattern**) Trả ra chuỗi ký tự đầu tiên phù hợp với điều kiện lọc. Xem thêm **str_match_all**.
`str_match(sentences, "(a|the) ([^ ]+)")`

Quản lý độ dài của chuỗi

 **str_length**(string) Số lượng ký tự có trong 1 chuỗi.
`str_length(fruit)`

 **str_pad**(string, width, side = c("left", "right", "both"), pad = " ") Chèn thêm ký tự để chuỗi có độ dài cố định.
`str_pad(fruit, 17)`

 **str_trunc**(string, width, side = c("right", "left", "center"), ellipsis = "...") Tính lược ký tự trong chuỗi.
`str_trunc(fruit, 3)`

 **str_trim**(string, side = c("both", "left", "right")) Cắt ký tự trắng bắt đầu hoặc kết thúc của chuỗi.
`str_trim(fruit)`

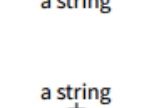
Biến đổi chuỗi

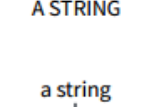
 **str_sub()** <- value. Thay chuỗi con bằng giá trị mới.
`str_sub(fruit, 1, 3) <- "str"`

 **str_replace**(string, **pattern**, replacement) Thay thế ký tự đầu tiên trong chuỗi thỏa mãn điều kiện.
`str_replace(fruit, "a", "-")`

 **str_replace_all**(string, **pattern**, replacement) Thay thế tất cả ký tự trong chuỗi thỏa mãn điều kiện.
`str_replace_all(fruit, "a", "-")`

 **str_to_lower**(string, locale = "en")¹ Chuyển sang chữ thường.
`str_to_lower(sentences)`

 **str_to_upper**(string, locale = "en")¹ Chuyển sang chữ hoa.
`str_to_upper(sentences)`

 **str_to_title**(string, locale = "en")¹ Chuyển sang định dạng tên.
`str_to_title(sentences)`

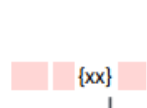
Gộp và tách chuỗi

 **str_c**(..., sep = "", collapse = NULL) Gộp nhiều ký tự thành một.
`str_c(letters, LETTERS)`

 **str_c**(..., sep = "", collapse = NULL) Gộp các ký tự/chuỗi trong một véc-tơ thành một chuỗi duy nhất.
`str_c(letters, collapse = "")`

 **str_dup**(string, times) Lặp lại chuỗi.
`str_dup(fruit, times = 2)`

 **str_split_fixed**(string, **pattern**, n) Tách một véc-tơ chuỗi với điều kiện và số lượng chuỗi con xác định trước.
`str_split_fixed(fruit, " ", n=2)`

 **glue::glue**(..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Tạo chuỗi từ chuỗi và sử dụng {} để thực hiện câu lệnh (nếu có).
`glue::glue("Pi is {pi}")`

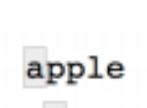
 **glue::glue_data**(.x, ..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Sử dụng dataframe để tạo chuỗi mới và dùng {} để thực hiện câu lệnh.
`glue::glue_data(mtcars, "{rownames(mtcars)} has {hp} hp")`

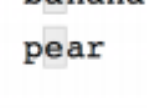
Sắp xếp ký tự

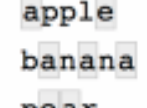
 **str_order**(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...) ¹ Trả ra kết quả là một véc-tơ thứ tự ký tự sau khi sắp xếp. `x[str_order(x)]`

 **str_sort**(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...) ¹ Sắp xếp lại vị trí ký tự.
`str_sort(x)`

Các hàm hữu ích khác

 **str_conv**(string, encoding) Thay đổi encoding của chuỗi. `str_conv(fruit, "ISO-8859-1")`

 **str_view**(string, **pattern**, match = NA) Xem kết quả HTML với ký tự đầu tiên thỏa mãn điều kiện lọc.
`str_view(fruit, "[aeiou]")`

 **str_view_all**(string, **pattern**, match = NA) Xem kết quả HTML với tất cả ký tự thỏa mãn điều kiện lọc.
`str_view_all(fruit, "[aeiou]")`

 **str_wrap**(string, width = 80, indent = 0, exdent = 0) Co ngắn độ dài của chuỗi.
`str_wrap(sentences, 20)`

¹ Xem thêm bit.ly/ISO639-1.

Những điều cần biết

Điều kiện lọc trong stringr sẽ được diễn dịch thành biểu thức chính quy (regular expressions – viết tắt là regex).

Trong R, ta viết biểu thức chính quy (regex) dưới dạng chuỗi ký tự trong dấu ngoặc kép hoặc ngoặc đơn.

Không phải tất cả các ký tự đều có thể thể hiện trực tiếp trong R dưới dạng chuỗi. Một số trường hợp phải thể hiện dưới dạng ký tự đặc biệt với ý nghĩa riêng.

Ký tự	Ý nghĩa
\\	\
\"	"
\n	Dòng mới

Thực hiện câu lệnh `?""` để xem thêm.

Do đó, bất cứ khi nào ký tự \ xuất hiện trong regex, ta phải viết dưới dạng hai dấu gạch sổ \ trong chuỗi thể hiện của regex.

Sử dụng hàm `writeLines()` để xem R thực hiện chuỗi với các ký tự đặc biệt như thế nào

```
writeLines("\\")
# \
```

```
writeLines("\\ is a backslash")
# \ is a backslash
```

Giải thích

Điều kiện lọc trong stringr được diễn dịch thành biểu thức chính quy (regex). Để thay đổi chế độ mặc định, ta có thể sử dụng các hàm sau:

`regex(pattern, ignore_case = FALSE, multiline = FALSE, comments = FALSE, dotall = FALSE, ...)` Điều chỉnh biểu thức chính quy, cho phép bỏ qua các điều kiện đặc biệt, cho phép R ghi chú bên trong một biểu thức chính quy.
`str_detect("I", regex("i", TRUE))`

`fixed()` Khớp các ký tự dưới dạng bytes, tuy nhiên có thể sẽ bị sót các ký tự có khả năng biểu diễn dưới nhiều hình thức khác nhau.
`str_detect("u0130", fixed("i"))`

`coll()` Khớp các ký tự dưới dạng bytes nhưng sử dụng locale để xác định chính xác ký tự.
`str_detect("u0130", coll("i", TRUE, locale = "tr"))`

`boundary()` Khớp với các ký tự phân biệt giữa câu, từ, dòng mới...
`str_split(sentences, boundary("word"))`

Biểu thức chính quy (regex) - Biểu thức chính quy hay regex là phương pháp chính xác để mô tả điều kiện lọc trong chuỗi

see <- function(rx) str_view_all("abc ABC 123\t!?\n", rx)

Ký tự	Regex	Ý nghĩa	Ví dụ
a (etc.)	a (etc.)	a (etc.)	see("a")
\\.	\\.	.	see("\\.")
\\!	\\!	!	see("\\!")
\\?	\\?	?	see("\\?")
\\\\	\\\\	\\	see("\\\\")
\\(	\\(	(	see("\\(")
\\)	\\)	)	see("\\)")
\\{	\\{	{	see("\\{")
\\}	\\}	}	see("\\}")
\\n	\\n	Xuống dòng	see("\\n")
\\t	\\t	Tab	see("\\t")
\\s	\\s	Khoảng trắng	see("\\s")
\\d	\\d	Số (\\D với điều kiện không phải số	see("\\d")
\\w	\\w	Chữ (\\W với điều kiện không phải chữ)	see("\\w")
\\b	\\b	Đường biên của một từ	see("\\b")
[[:digit:]]	[[:digit:]]	Số	see("[[:digit:]]")
[[:alpha:]]	[[:alpha:]]	Chữ	see("[[:alpha:]]")
[[:lower:]]	[[:lower:]]	Viết thường	see("[[:lower:]]")
[[:upper:]]	[[:upper:]]	Viết hoa	see("[[:upper:]]")
[[:alnum:]]	[[:alnum:]]	Số hoặc chữ	see("[[:alnum:]]")
[[:punct:]]	[[:punct:]]	Dấu	see("[[:punct:]]")
[[:graph:]]	[[:graph:]]	Chữ, số hoặc dấu	see("[[:graph:]]")
[[:space:]]	[[:space:]]	Dấu cách	see("[[:space:]]")
[[:blank:]]	[[:blank:]]	Khoảng trắng	see("[[:blank:]]")
.	.	Tất cả ký tự trừ dòng mới	see(".")



Điều kiện alt <- function(rx) str_view_all("abcde", rx)

Điều kiện	regex	Ý nghĩa	Ví dụ
	ab d	Hoặc	alt("ab d")
	[abe]	Một trong	alt("[abe]")
	^[abe]	Tất cả trừ	alt("^[abe]")
	[a-c]	Trong khoảng	alt("[a-c]")

Điều kiện chặn anchor <- function(rx) str_view_all("aaa", rx)

Điều kiện chặn	regex	Ý nghĩa	Ví dụ
	^a	Bắt đầu chuỗi	anchor("^a")
	a\$	Kết thúc chuỗi	anchor("a\$")

Tìm mở rộng look <- function(rx) str_view_all("bacad", rx)

Tìm mở rộng	regex	Ý nghĩa	Ví dụ
	a(=?c)	Tìm a theo sau là c	look("a(=?c)")
	a(?!c)	Tìm a với ký tự đằng sau không phải là c	look("a(?!c)")
	(?<=b)a	Tìm a với ký tự đằng trước là b	look("(?<=b)a")
	(?<!b)a	Tìm a với ký tự đằng trước không phải b	look("(?<!b)a")

Đếm số lượng quant <- function(rx) str_view_all("a.aa.aaa", rx)

Đếm số lượng	regex	matches	example
	a?	Không hoặc một	quant("a?")
	a*	Không hoặc nhiều	quant("a*")
	a+	Một hoặc nhiều	quant("a+")
	a{n}	Chính xác n	quant("a{2}")
	a{n,}	n hoặc nhiều hơn	quant("a{2,}")
	a{n,m}	Giữa n và m	quant("a{2,4}")

Nhóm ref <- function(rx) str_view_all("abbaab", rx)

Sử dụng dấu ngoặc để tạo nhóm

regex	Điều kiện	Ví dụ
(ab d)e	sets precedence	alt("(ab d)e")

Sử dụng dấu gạch sổ để lặp lại nhóm trong ngoặc đơn. Số thứ tự chính là thứ tự của nhóm.

Ký tự (câu lệnh)	regex (Ý nghĩa)	Diễn giải (Điều kiện lọc)	Ví dụ (Kết quả tương tự như ref("abba"))
\\1	\\1 (etc.)	Nhóm đầu tiên.	ref("(a)(b)\\2\\1")